

Creating the content page; ii) Writing the code for the mobile application.

Creating the content page: In Xamarin, the content page is the most commonly used page. This is used to set the application's label and layout properties. The source code for a content page can be written in the *MainActivity.cs* file. This file is available under the *open solution explorer* menu. The screenshot for the same is shown in Figure 5.

Writing code for the mobile application: After creating the content page, open the *App.xaml* source file and write the code for the mobile application. In this test example, we have created a 'Hello World' application. Once the code has been

written, build and run the application code using Android Emulator. A successful build and run step will lead to the output, as shown in Figure 6.

In the last decade, Xamarin has emerged as the most preferred mobile app development framework. This is because it provides a platform to quickly

build the mobile application with cost-effectiveness. Its popular features are integrated testing, easy maintenance and a fast development cycle.

However, the Xamarin platform may not be a good choice for developing highly complex user interfaces and system dependency applications.  

References

- [1] <https://visualstudio.microsoft.com/xamarin/>
- [2] <https://github.com/xamarin>
- [3] <https://dotnet.microsoft.com/apps/xamarin>

By: Dr Aditya Bhardwaj

The author is a B. Tech, M. Tech and PhD in CSE. He works as assistant professor in the computer science and engineering department at KIET Group of Institutions, Ghaziabad. He has experience in cloud computing and Big Data technology.

Continued from page 50...

2. QUIC is all about pushing the 'network stack' (transport protocol implementation) from the kernel to user space.
3. It is different from http2 because:
 - a. It is on UDP, so is lighter than TCP. Even vis a vis the latest congestion techniques of TCP (like Cubic, reference: https://en.wikipedia.org/wiki/CUBIC_TCP), UDP is minimalistic and has often proven to be better.
 - b. Http2 also multiplexes multiple streams through the same TCP connection.
 - c. However, if the 'head' frame suffers a packet loss, the other frames get stuck – that's http2 Head of Line (HOL) congestion.
 - d. QUIC solves the problem differently:
 - Each overlay data stream has its own end-to-end flow state.
 - Delay in one data stream does not affect another.
4. Http3 is now with the Internet

Browser	Version implemented (Disabled by default)	
Chrome	Stable build (79)	December 2019
Firefox	Stable build (72.0.1)	January 2020
Safari	Safari Technology Preview 104	April 2020
Edge	Edge (Canary build)	January 2020

Table 2: Http3 browser support

Engineering Task Force (IETF) for standardisation.

5. It is supported, often under a flag, by most browsers (Table 2).  

References

- [1] <https://queue.acm.org/detail.cfm?id=2555617>
- [2] <https://httparchive.org/reports/state-of-the-web?start=earliest&end=latest&view=list>
- [3] <https://opensourceforu.com/2020/03/the-evolution-of-web-protocols-2/>
- [4] <https://www.igvita.com/>
- [5] https://hpbn.co/?utm_source=igvita&utm_medium=referral&utm_campaign=igvita-homepage
- [6] https://en.wikipedia.org/wiki/Transport_Layer_Security
- [7] https://en.wikipedia.org/wiki/CUBIC_TCP
- [8] <https://blog.apnic.net/2019/03/04/a-quick-look-at-quic/>
- [9] <https://twitter.com/SimmerVigor>

By: Pradip Mukhopadhyay

The author has 20 years of experience across the stack—from low level system programming to high level GUI. He is a FOSS enthusiast. He is currently working for NetApp, Bengaluru.